Karpathy Modelleri FPGA'de · konuşma notları

Bu belge ne? Sunum sırasında elinde tutman için kısa notlar: her slayt için ana mesaj ve söylenecek sayılar, bir zaman planı ve hocanın sorabileceği sorular. Slayt slayt tam konuşma metni ayrı dosyada: `konusma_metni.pdf`.

Zaman planı (~45 dakika)

Bölüm	Slaytlar	Süre
Giriş, özet, yol haritası	1–3	2 dk
1 · Hedef ve kart	4–6	3 dk
2 · Model ailesi	7–10	4 dk
3 · Integer arithmetic	11–17	7 dk
4 · Compiler	18–20	3 dk
5 · Engine	21–25	5 dk
6 · Kart sistemi	26–30	4 dk
7 · Low-bit weight'ler (LittleBit, Larq)	31–36	5 dk
8 · Verification	37–40	4 dk
9 · Place and route ve hız	41–45	4 dk
10 · Sonuç, katkılar	46–51	5 dk

20 dakikalık kısa sürüm: 1, 2, 4, 5, 7, 10, 12, 13, 14, 17, 21, 22, 26, 28, 31, 33, 34, 36, 37, 38, 42, 43, 45, 46, 48, 51. Atlanan slaytlarda bir cümleyle geç.

Slayt slayt ana mesajlar

Giriş

1 · Kapak — Bölüm 2: implementasyon. Terimler İngilizce. **2 · Özet** — Dört soru, dört cevap: FPGA hesabı bozmuyor (18/18 bit-exact, 0 fark); integer'a geçişin bedeli ≤ %1,6 loss (names ≤ %0,3); 16/18 model chip'e sığıyor ve timing'i karşılıyor; stories260K 496 token/s. CPU (M1, run.c, -Ofast; -O3 ile 6 718): 14 397 token/s, ~222 273 cycle/token; FPGA 50 397 cycle/token → clock başına 4,4× (mutlak hızda CPU 29× önde). **3 · Yol haritası** — PyTorch'tan chip'e dokuz adım; son üçü (turuncu) bu aşamanın yenileri.

1 · Hedef

4 · Amaç — Host-driven düzen değil, bağımsız kart. Üç kural: yalnız inference, yalnız Karpathy modelleri, verification Mac'te. **5 · Kart** — XC7A100T: 63 400 LUT, 126 800 FF, 135 BRAM (~607 KB), 240 DSP, CPU yok. DDR2 bilerek kullanılmıyor. **6 · Kısıtlar** — İşlemci yok, float yok, external memory

yok, tek clock. stories260K: 254 KB weight + 80 KB KV cache < 607 KB; GPT-2 124M 124 MB isterdi.

2 · Model ailesi

7 · Aile — makemore (5), minGPT/nanoGPT (gpt2), microgpt, llama2.c, nanochat. Hepsi names.txt, ~50 bin parametre. **8 · Yapı taşları** — Hepsi aynı birkaç op'un birleşimi; op sayısı bigram 1 … stories260K 73. **9 · stories260K** — Tek hazır checkpoint: 64 boyut, 5 layer, 8/4 head, 512 token. Float 1,400 → INT8 1,422. ts_ *sürümleri bizim; iki tabloyu doğrudan karşılaştırma. 10 · Ekrandan** — Kareler pinlerden yakalandı, 0 pixel fark. Status line'daki tok/s 50 MHz varsayımıyla.

3 · Integer arithmetic

11 · Neden integer — INT8 MAC = 1 DSP, 1 cycle. Örnek dot product 4 411. stories260K float32'de 1 040 KB, BRAM'e sığmaz. **12 · Quantization** — S = 0,268/127 = 0,00211; 0,0372 → 18; hata ≤ S/2. Weight per-channel, activation %99,99 yüzdeliği. **13 · Requant** — $M \approx m_0 \cdot 2^{-sh}$. Örnek: m_0 = 45 979, sh = 25, acc = 5 321 → 7 (float 7,29). **14 · Softmax** — Max çıkar, 2'nin kuvveti = shift + 256 entry'lik tablo, tek bölme. Örnek 0,607 / 0,287 / 0,106, float ile aynı. **15 · Norm** — SUM, VAR, SQRT, DIV, OUT; Samuelson: overflow yok; γ, β fold edilir. NORM gpt2'de %14, nanochat'te %33. **16 · Tablolar** — INT8 input'lu fonksiyon = 256 satır, yaklaşımsız (GELU, SiLU, tanh, σ, $ReLU^2$). **17 · Kalite** — Δ loss names'te ≤ 0,0055, stories260K'da 0,022. argmax aynılığı %92–100.

4 · Compiler

18 · Akış — Model → IR (< 10^{-4}) → katlama → kalibrasyon → IntRunner (altın referans) → .mem. **19 · Microcode** — gpt2 ile llama'nın block'u yan yana; fark RoPE, GQA, SwiGLU. **20 · Memory** — stories260K BRAM'in ~%71'i: weight 256 KB, KV 80 KB, tablolar 68 KB. Weight'ler bitstream'in içinde.

5 · Engine

21 · zoo_core — Fetch, decode, unit'ler, 11 buffer; 16 op türü. **22 · GEMV** — 8 lane, input broadcast edilir, 64 bitlik word 8 weight. cycle ≈ ⌈n_out/8⌉ · n_in (gpt2 fc 512, stories w1 1 408). **23 · Attention** — Her geçmiş token head başına 2 cycle: 5 × 8 × 2 = 80 cycle/token eğim; ölçüm 80,0. **24 · Profil** — makemore'da GEMV %94–96; nanochat'te NORM %33; stories260K'da ATTN %37. **25 · SPECIALIZE** — Kullanılmayan unit silinir: bigram 5 855 → 1 529 LUT.

6 · Kart sistemi

26 · Sistem — Döngü, sampler, detokenizer, terminal, VGA; ekran yazımı sonraki adımla örtüşür. **27 · Generation loop** — G_STEP … G_PICK; sampler ≈ 3V cycle (stories'te ~1 500, %3). **28 · Sampling** — Bölmesiz: thr = (r · Σe) ≫ 32. Örnek: 0,7775 / 0,2225, float ile binde bir. **29 · Ekran** — Detokenizer iki ROM; word wrap; 640 × 480, 8 × 16 font. **30 · Kullanım** — Switch'ler, button'lar; hash h ← 31h + tok; LED16 yeşil = Python ile aynı.

7 · Low-bit weight'ler

31 · Neden — 1 bit: memory 8 kat küçük, çarpma yok (12 + 7 + 30 = 49). Hocanın iki kaynağı: LittleBit, Larq. FPGA'de training yok. **32 · Yöntemler** — int8, int4, tern, bin, LittleBit; STE ve KD. Larq PyTorch'ta yeniden yazıldı. **33 · Sonuç** — gpt2'de binary (1,38 BPW, 2,029) en iyi; mlp'de LittleBit (1,18 BPW, 2,075) binary'yi (2,160) geçiyor. INT4 her yerde float kadar iyi. **34 · Neden** — BPW

formülü; 32 × 32'de yalnız scale'ler 1,00 BPW. 0,1 BPW ancak 4096²'de. **Hocanın 1. araştırma sorusunun cevabı. 35 · GEMVB** — 64 lane, DSP yok, 20 bit accumulator; SCALE ile W·diag(γ) = diag(a)·S·diag(g). **36 · Kartta** — gpt2-bin 1,7×, mlp-lb 3,6×, stories-bin 1,8× hız; BRAM %43–64 az; loss bedeli +0,03 / +0,16 / +0,64.

8 · Verification

37 · Zincir — 8 halka; kart olmadan neden yeterli: tek clock, netlist doğru + STA. **38 · Tam kart** — Testbench kullanıcı gibi; yalnız pinler; 18/18, 11 241 token. **39 · Random** — 655 koşu, 16 corner case, 0 X; iki RTL hatası (kaybolan button, reset'siz counter). **40 · Netlist** — Tool bug'ları: sv2v signedness, Yosys wreduce+peepopt, Yosys DSP sign extension.

9 · Place and route ve hız

41 · Akış — Yosys → nextpnr → FASM → X-Ray → .bit. Sorunlar: SDK, PACKAGE_PIN, 50 MHz'te durma, DSP. **42 · Fmax** — bigram, mlp, rnn 50 MHz'te (66–71); transformer'lar 26–31 → 25 MHz. ts_gpt2, ts_microgpt yerleşemedi (RAMB18). Fmax seed ile oynuyor. **43 · Critical path** — Oku → çarp → yaz tek cycle'da; stories260K 36,7 ns, %73 routing. Çare: pipeline. **44 · Kaynaklar** — En büyük model LUT'un %31'i; DSP 0 (multiplier'lar LUT'ta). **45 · Hız** — stories260K 496 token/s (256 token yarım saniye); bigram 318 bin; mlp-lb 13 bin.

10 · Sonuç

46 · Cevap — Kaybın tek kaynağı quantization (1,400 → 1,422); FPGA bit-exact, 0 kayıp ekliyor. CPU (M1, run.c, -Ofast; -O3 ile 6 718): 14 397 token/s, ~222 273 cycle/token; FPGA 50 397 cycle/token → clock başına 4,4× (mutlak hızda CPU 29× önde). Enerji ölçülmedi, bunu açıkça söyle. **47 · Karşılaştırma** — names'te en iyi loss gru, en iyi denge rnn/mlp (50 MHz); LittleBit hızı loss ile ödüyor. **48 · Katkılar** — Beş katkı; literatürden alınanları açıkça söyle. "Biz icat ettik" değil, "birleştirdik ve ölçtük". **49 · Sınırlar** — Kart yok, Fmax yaklaşık, kapasite ~300 bin parametre, eksik parçalar. **50 · Sonraki adımlar** — Pipeline → 50 MHz; netlist simülasyonu; DSP ve memory layout; gpt-nano; düşük bit. **51 · Özet** — Dört cümle, teşekkür.

Olası sorular ve kısa cevaplar

Kart olmadan çalıştığından nasıl eminsiniz? Tasarım tek clock'lu ve synchronous. Netlist'in functional doğruluğunu simülasyonla (gerçek üst modül, her pin, her cycle), her path'in bir periyoda sığdığını timing analysisyle gösterdik. Endüstride de üretimden önceki kanıt bu ikisi. Tek boşluk: placement'tan sonraki circuit'in simülasyonu; nextpnr onu Verilog olarak yazamıyor.

Neden Vivado kullanmadınız? Sonuçlar Vivado'da farklı olur mu? Vivado macOS'ta çalışmıyor; akış tamamen open-source. nextpnr'ın delay modeli Project X-Ray'den, Vivado'nunki farklı olabilir. Vivado DSP'leri doğru eşlediği için orada Fmax'ın daha yüksek, LUT'un daha düşük çıkmasını bekleriz. Vivado betiği hazır (`make nexys-bit`).

Dizüstü CPU 29 kat hızlıysa FPGA'nin anlamı ne? Mutlak hızda M1 önde (14 397 ve 496 token/s), çünkü clock'u 128 kat yüksek ve çekirdeği çok daha büyük bir silikon. Token başına cycle'da FPGA 4,4 kat verimli (50 397 ve ~222 bin). Amacımız laptop'ı geçmek değil: CPU'suz, OS'suz, deterministik ve bit-exact bir sistem. Pipeline ve DSP ile clock 50 MHz ve üstüne çıkar; enerji ölçülmediği için token/J iddiası yok.

Neden 25 MHz? 50'ye nasıl çıkar? Engine buffer'dan okuma, çarpma ve geri yazmayı tek cycle'da yapıyor; çarpmalar LUT'ta olunca yol 26–37 ns. Okuma ile hesap arasına bir register koyarsak yol ikiye bölünür; hedef 50 MHz, token/s iki kat.

Neden DSP kullanılmıyor? Yosys 0.69, requant multiply'ını DSP48E1'e eşlerken sign extension'ını düşürüyor; negatif değerler yanlış çıkıyor. Netlist simülasyonu bunu yakaladı. Doğru çalışan netlist için multiplier'ları LUT'a koyduk (-nodsp).

Neden INT4 değil de INT8? INT4 float kadar iyiydi. INT4'ün donanımdaki tek kazancı memory'si yarıya indirmek; modellerimiz BRAM'e INT8'le zaten sığıyor. Hesap tarafında binary daha büyük kazanç sağlıyor (multiplier'sız).

LittleBit neden makaledeki gibi iyi çıkmadı? Makale 1,3 milyar parametre ve üstünü inceliyor. Scale vector'lerinin bedeli layer genişliği d ile 1/d azalıyor; 32 genişliğinde yalnız scale'ler 1 BPW. Yani küçük modellerde bitler rank'a değil scale'lere gidiyor. Geniş input'lu layer'larda (mlp, d_in = 256) LittleBit binary'yi geçiyor; bu, formülün öngördüğü şey.

Larq'ı neden doğrudan kullanmadınız? Larq TensorFlow kütüphanesi; quantizer'ları birkaç satırlık fonksiyonlar. Belgelenen tanımlarına göre PyTorch'ta yazınca modeller, training ve integer akış aynı kaldı. Gerçek paketle sayısal karşılaştırma henüz yapılmadı.

Bit-exact neden önemli? Float ile neden yaklaşık? Hardware ve Python referansı aynı integer işlemleri yapıyor; çıktıların aynı olmaması bir hata demek. Float ile integer ise farklı aritmetik; aradaki farkı loss ile ölçüyoruz (≤ 0,022 nat).

stories260K'nın INT8 kaybı neden diğerlerinden büyük? Model bizim kalibrasyonumuzu bilmeden float olarak eğitildi ve 5 layer derinliğinde; hatalar layer'lar boyunca birikiyor. Yine de fark %1,5.

Bu modeller çok küçük; büyük modeller ne olacak? Chip içi 607 KB ~300 bin parametreye yetiyor. Daha büyük modeller için kartın 128 MB DDR2'sinden weight akıtan bir mimari gerekir; bu, memory bant genişliğinin baskın olduğu farklı bir problem.

GPU ya da CPU ile karşılaştırdınız mı? Güç tüketimi? Hayır, ikisi de ölçülmedi. Bu tasarımın amacı en yüksek hız değil, mimarilerin aynı zeminde karşılaştırılması ve bağımsız, bit-exact bir hardware. Güç için Vivado'nun güç tahmini ya da kartta ölçüm gerekir; sonraki adımlar arasında.

ts_gpt2 ve ts_microgpt neden yerleşemedi? Yosys bu iki modelin tablolarını çok sayıda küçük RAMB18 bloğuna bölüyor; BRAM yerlerinin ~%85'i doluyor ve nextpnr'ın placer'ı iki farklı seed ile 40 dakikada ilerlemedi. Çözüm tabloların memory layout'unu değiştirmek. İkisi de simülasyonda bit-exact.

Projenin özgün katkısı ne? Slayt 48: eşit bütçe karşılaştırması (en savunulabilir), tek engine'de 10 bit-exact model, bağımsız integer kart, ispatlı sınırlar ve dört tool bug'ı, LittleBit'in küçük ölçekteki sınırı. Requant, integer softmax, binary weight'ler gibi teknikler literatürden.

Sampling nasıl integer yapılıyor? min-p ne? Softmax'la aynı 2^(−k/256) tablosu; probability normalize edilmiyor, random number toplamla çarpılıp 32 bit shift ediliyor. min-p, en olası token'ın belli bir oranından (1/32) az olasılıklı token'ları atıyor.

Neden bu kart? İşlemcisi olmadığı için modelin gerçekten FPGA'de çalıştığı bir tasarıma zorluyor; 607 KB BRAM stories260K'ya yetiyor; VGA, switch ve button'lar bağımsız bir demo için yeterli.